1. Objective

Joins combine related tables. Subqueries help compare values or check whether related records exist.

2. Join Passenger, Booking and Flight

```
SELECT b.Booking_ID, p.Name, f.Flight_Number, b.Seat_ID, b.Booking_Status\nFROM Booking b\nJOIN
Passenger p ON b.Passenger_ID=p.Passenger_ID\nJOIN Flight f ON b.Flight_ID=f.Flight_ID\nORDER BY b.
Booking_ID;
```

Booking_ID	Passenger	Flight_Number	Seat_ID	Status
5001	Aarav Sharma	AI101	103	Confirmed
5002	Diya Kumar	AI101	104	Confirmed
5003	Rahul Menon	AI202	202	Confirmed
5004	Meera Iyer	AI202	203	Pending
5005	Arjun Rao	AI303	105	Confirmed
5006	Sara Khan	AI404	202	Confirmed

This report shows each booking with passenger name and flight number.

3. Join Departure and Arrival Airports

```
SELECT f.Flight_Number, d.City AS Departure_City, a.City AS Arrival_City, f.Status\nFROM Flight f\nJOIN
Airport d ON f.Departure_Airport=d.Airport_ID\nJOIN Airport a ON f.Arrival_Airport=a.Airport_ID\nORDER
BY f.Flight_Number;
```

Flight_Number	Departure_City	Arrival_City	Status
AI101	Bengaluru	Chennai	Scheduled
AI202	Chennai	Delhi	Scheduled
AI303	Delhi	Mumbai	Delayed
AI404	Mumbai	Bengaluru	Scheduled

Airport is joined twice because a flight has both a departure and arrival airport.

4. Subquery: Payments Above Average

```
SELECT Payment_ID, Booking_ID, Amount\nFROM Payment\nWHERE Amount > (SELECT AVG(Amount) FROM Payment)\
nORDER BY Amount DESC;
```

Payment_ID	Booking_ID	Amount
9005	5005	7800.00
9003	5003	6200.00
9004	5004	6200.00

The subquery calculates the average payment (5733.33). The outer query shows amounts above that average.

5. Subquery: Passengers with Bookings

```
SELECT Passenger_ID, Name FROM Passenger p\nWHERE EXISTS (SELECT 1 FROM Booking b\nWHERE b.
Passenger_ID=p.Passenger_ID)\nORDER BY Passenger_ID;
```

Passenger_ID	Name
1	Aarav Sharma
2	Diya Kumar
3	Rahul Menon
4	Meera Iyer
5	Arjun Rao
6	Sara Khan

6. Summary by Flight

```
SELECT f.Flight_Number, COUNT(b.Booking_ID) AS Booking_Count,\n SUM(py.Amount) AS Recorded_Amount\nFROM
Flight f LEFT JOIN Booking b ON f.Flight_ID=b.Flight_ID\nLEFT JOIN Payment py ON b.Booking_ID=py.
Booking_ID\nGROUP BY f.Flight_ID, f.Flight_Number\nORDER BY f.Flight_Number;
```

Flight_Number	Booking_Count	Recorded_Amount
AI101	2	9000.00

Flight_Number	Booking_Count	Recorded_Amount
AI202	2	12400.00
AI303	1	7800.00
AI404	1	5200.00

LEFT JOIN keeps flights even if they have no bookings. Recorded amounts include pending payments and are not necessarily collected revenue.

7. Conclusion

The reports demonstrate multi-table joins and subqueries for bookings, routes, payments and flight summaries.